```
class some_data
{
    int a;
    std::string b;
public:
    void do_something();
};

class data_wrapper
{
private:
    some_data data;
    std::mutex m;

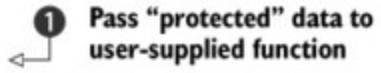
public:
    template<typename Function>
    void process_data(Function func)
    {
        std::lock_guard<std::mutex> l(m);
        func(data);
    }
};

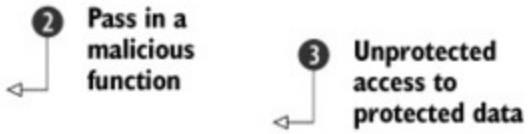
some_data* unprotected;

void malicious_function(some_data& protected_data)
{
    unprotected=&protected_data;
}

data_wrapper x;

void foo()
{
    x.process_data(malicious_function);
    unprotected->do_something();
}
```





In this example, the code in `process_data` looks harmless enough, nicely protected with `std::lock_guard`, but the call to the user-supplied function `func` ❶ means that `foo` can pass in `malicious_function` to bypass the protection ❷ and then call `do_something()` without the mutex being locked ❸.

Fundamentally, the problem with this code is that it hasn't done what you set out to do: mark all the pieces of code that access the data structure as *mutually exclusive*. In this case, it missed the code in `foo()` that calls `unprotected->do_something()`. Unfortunately, this part of the